

LESSONS

1. Integrated Development Environment Creation of project file with “Simple Application”
Work with screen resource Editors
2. Work with the resource and controls editors
3. Application with Dialog Box Application with multifunctional Menu and processing of different users answers
4. Application with Dialog Box with different Dialog Controls-1 (Calculation using integer and real data)
5. Application with Dialog Box with different Dialog Controls 2 (Combo, List end Group Controls)
6. Application with Dialog Box (Mouse Mouving Timer, Progress bar)
7. Graphical Application –Introduction
8. Graphical Application – Moving with mouse (Drag&Drop)
9. Text Editor (WIN32 and using MFC)
10. Workin with Files Open and Save
11. MFC and ActiveX controls (Calendar).
12. Management od Databases ODBC-interface. App Wizard for data bases application.

LESSON 1

INTEGRATED DEVELOPMENT ENVIRONMENT CREATION OF PROJECT FILE WITH "SIMPLE APPLICATION" WORK WITH SCREEN RESOURCE EDITORS

THEORY

Integrated Development Environment – (IDE)

IDE of MS Visual C++ provides programmer with the following instruments :

- **Text editor** - for **source** code creation;
- Tools for discovery of programming errors - **debugger**;
- **Compiler** of source and resource files and **Linker**;
- **help** system - Microsoft Developer Network (**MSDN**);
- Tools for program /code/ **optimization**;
- **ClassWizard** a device for application classes creation and management ;
- **AppWizard (AFC)** a device for skeleton applications creation and editing
- **libraries**: Object Link Embedding (**OLE**), Runtime Libraries (**RTL**); Active Template Library (**ATL**); *.dll (Dynamic Link Library)

Files, created in Environment of Microsoft Visual C++

The environment uses the following folders and files:

With the creation of a new project, a folder with the **same name** as that of the project is created. The folder contains a folder with the name **Debug** and can contain the following types of files:

- *.dsw – Project Workspace ;
- *.dsp – Project File ;
- *.cpp – C++ source file – a file with primary code /source/ of the programs;
- *.h, – header file – a file with primary code of the used libraries;
- resource.h – contains the name of the resources ;
- *.rc - Resource Template – a file with a template of the resources;
- ReadMe.txt -Text Document ;
- *.asp – ASP File- Active Server Pages;
- *.ico ; smal.ico – Icon File – images of icons;
- *.ncb – NCB File
- *.html – HTML Documents;
- *.cur - cursor image;
- *.exe - respectively executable transferable program – application;
- *.dll - Application, and dynamic program library.

INSTRUCTION

1. Start MS Visual C++

2. Create new project: Click on **File | New | Projects**.

- Choose project type: **Win32 Application**. Insert of project **name**.
- Select from menu the ability: **A typical "Hellow Word"**.

There is possibility for (**An Empty project**) and (**A simple win32 application**) for application, which run end exit.

3. Examine abilities of next menu functions:

- **Project | Settings**
- **Tools | Options**

4. Tray to use functions - **Insert, Add, Modify, Delete** of files and resources - Select object **| Insert or Edit or Delete**

5. Create of application and work with Builder:

- Click on **Build | Compile**;

- Click on **Build | Start Debug;**
 - Click on **Build | Execute** Application
6. Examine elements of whole project. Use the tools for working with workspace components (**Class View, File View, Resource View**).
- Click on **Class View** and select components;
 - Click on **Resource View** and select components;
 - Click on **File View** and select components;
7. Close Visual C++. Start MS Visual C++ again. Open Workspace of your project
- (Click on **File | Open Workspace** | select file *.dsw

TASKS

1. Find the resource, which determinate text "Hallow Word".
2. Change this text by "Hallow my friend".
3. Execute the modified application.
4. Make a new application icon. Change the place of dialog and controls.
5. Close application and Visual C++.
6. Start IDE again and load workspace of your project.
7. Add new functions to menu – for example : New function Level1 | Level2 | Level3
8. Execute the modified application.
9. Pay attention on type of files and their contents.
10. Close application and Visual C++.

LESSON 2

WORK WITH THE RESOURCE AND CONTROLS EDITORS

APPLICATION WITH DIALOG BOX

THEORY

Resource

- Accelerators - Tables with combinations of keys
- Dialog-boxes
- Icons – For the creation and edition of icons, it uses tools which are grouping in section Colors and Graphics.
- Menu –
- String Table – Table with texts
- ToolBar – Tool band
- BitMap - Bite images for the creation and edition of icons it uses tools, which are grouping in, section Colors and Graphics.
- Cursor – Cursors for the mouse
- Version – information for the program version
- HTML – It has created using a text edition and section tools.

Methods, used for work with dialog boxes

New Methods in the lesson example are:

SetDlgItemText(hDlg, Control Identification , "some text")- initializes of the control;
for example - SetDlgItemText(hDlg, IDC_EDIT1, "Example1");

EndDialog(hDlg, LOWORD(wParam)) - end of dialog

int MessageBox(hDlg, "Message", "Title of box", Buttons| Standard Windows) – displays a message according the parameterr:

- **Buttons** : {MB_OK, MB_OKCANCEL, MB_YESNO, MB_YESNOCANCEL, MB_RETRYCANCEL, MB_ICONEXCLAMATION }
- **Standard Windows**: { MB_ICONERROR, MB_ICONSTOP, MB_ICONINFORMATION, MB_ICONQUESTION }
for example - MessageBox(hDlg, "Message Box", "MSG", MB_YESNOCANCEL | MB_ICONERROR);
MessageBox(hDlg, "No", "MSG", MB_OK);
int x; x = MessageBox(hDlg, "Liub4oslav", "Popivanoslav", MB_YESNOCANCEL | MB_ICONQUESTION);

GetDlgItemText(hDlg, Control Identification , text variable , text size) – insert data
for example - char buf[100]; GetDlgItemText(hDlg, IDC_EDIT1, buf, 100);

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project **HW** (Click on **File | Open Workspace** | select file HW.dsw

2. Look at the following component of the application carefully:

- Choose Resource View and write in your notebook the next fields contains.

Menu: ID:

Function from the menu - **New Ability:** ID:.....**Caption:**

Prompt:.....

Dialog Box: ID: IDD_OUR

Caption: **Menu:** **XPos:**..... **YPos:**...

Edit Box and user Button in Dialog Box IDD_OUR

Edit Box: **Button Get** : **Button Press** :

- Choose File View end open file HW.cpp
- Find in program text **All identifications name.**

The main points are: in function **WndProc** (case IDM_NA: DialogBox...) and a new function.

3. Find and examine a new function, which is used in Dialog box IDD_OUR.

The function name is **FirstOurProg**.

You must find the name of this function in three places in the file. These 3 rows are:

- LRESULT CALLBACK **FirstOurProg**(HWND, UINT, WPARAM, LPARAM);
WinMain.....
WndProc.....
- DialogBox(hInst, (LPCTSTR)IDD_OUR, hWnd, (DLGPROC) **FirstOurProg**);
.....
- LRESULT CALLBACK **FirstOurProg** (HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam).....

4. Build and execute the application. Examine the ability of the new dialog box.

5. Concentrate your attention on the contents of the sub-program **FirstOurProg**. There are used the following important methods, which are connected with different control elements.

6. Note the process of messages in **case WM_COMMAND**:

There are constructions which process the pressing of the relevant button:

if (LOWORD(wParam) == IDOK || LOWORD(wParam) == IDCANCEL) {.....} // for dialog box

else if(LOWORD(wParam)== IDC_PRESS) {.....} // for button with ID: IDC_PRESS

else if(LOWORD(wParam)== IDC_GET) { }// for button with ID: IDC_GET

TASK

1. 1. make copy in a new folder of lesson example.
2. Make following changes in your copy:
 - Change the next ID-s: IDM_NA with IDM_NNAA, IDD_OUR with IDD_YOUR, IDC_EDIT1 with IDC_EDIT22;
 - Change the place of dialog and controls;
 - Insert new controls in current dialog;
 - Insert some new resources
3. Insert corresponding changes in the source code HW.cpp according to point 1. Compile, Build end Execute new application.
4. Insert 5 corrections (messages, titles, buttons, standard windows) in procedure **FirstOurProg** according your choice. Test end execute the new application.

LESSON 3

APPLICATION WITH MULTIFUNCTIONAL MENU AND PROCESSING OF DIFFERENT USERS ANSWERS

THEORY

Methods, used for work with menu

New Methods in the lesson example are:

GetMenu – retrieves the window's menu;

GetMenuState- retrieves the value if the corresponding caption exists.

UINT GetMenuState(UINT *nID*, UINT *nFlags*) const;

The value 0xFFFFFFFF if the specified item does not exist. If *nID* identifies a pop-up menu, the high-order byte contains the number of items in the pop-up menu and the low-order byte contains the menu flags associated with the pop-up menu.

MF_CHECKED Acts as a toggle with **MF_UNCHECKED** to place the default check mark next to the item. When the application supplies check-mark bitmaps, the “check mark on” bitmap is displayed. **MF_DISABLED** Disables the menu item so that it cannot be selected but does not dim it. **MF_ENABLED** Enables the menu item so that it can be selected and restores it from its dimmed state. Note that the value of this constant is 0; an application should not test against 0 for failure when using this value.

MF_GRAYED Disables the menu item so that it cannot be selected and dims it.

MF_MENUBARBREAK Places the item on a new line in static menus or in a new column in pop-up menus. The new pop-up menu column will be separated from the old column by a vertical dividing line. **MF_MENUBREAK** Places the item on a new line in static menus or in a new column in pop-up menus. No dividing line is placed between the columns.

MF_SEPARATOR Draws a horizontal dividing line. Can only be used in a pop-up menu. This line cannot be dimmed, disabled, or highlighted. Other parameters are ignored.

MF_UNCHECKED Acts as a toggle with **MF_CHECKED** to remove a check mark next to the item. When the application supplies check-mark bitmaps, the “check mark off” bitmap is displayed. Note that the value of this constant is 0; an application should not test against 0 for failure when using this value.

Parameters: *nID* Specifies the menu item ID, as determined by *nFlags*.

nFlags Specifies the nature of *nID*. It can be one of the following values:

- **MF_BYCOMMAND** Specifies that the parameter gives the command ID of the existing menu item. This is the default.
- **MF_BYPOSITION** Specifies that the parameter gives the position of the existing menu item. The first item is at position 0.

CheckMenuItem – insert check or uncheck in corresponding caption;

UINT CheckMenuItem(UINT *nIDCheckItem*, UINT *nCheck*);

Return Value -The previous state of the item: **MF_CHECKED** or **MF_UNCHECKED**, or 0xFFFFFFFF if the menu item did not exist.

Parameters: *nIDCheckItem* - Specifies the menu item to be checked, as determined by *nCheck*.

nCheck Specifies how to check the menu item and how to determine the item's position in the menu. The *nCheck* parameter can be a combination of **MF_CHECKED** or

MF_UNCHECKED with **MF_BYPOSITION** or **MF_BYCOMMAND** flags. These flags can be combined by using the bitwise OR operator. They have the following meanings:

MF_BYCOMMAND Specifies that the parameter gives the command ID of the existing menu item. This is the default. **MF_BYPOSITION** Specifies that the parameter gives the

position of the existing menu item. The first item is at position 0. **MF_CHECKED** Acts as a toggle with **MF_UNCHECKED** to place the default check mark next to the item.

MF_UNCHECKED Acts as a toggle with **MF_CHECKED** to remove a check mark next to the item.

EnableMenuItem – set enabled or grayed corresponding caption ;

UINT EnableMenuItem (UINT *nIDEnableItem*, UINT *nEnable*);

Return Value - Previous state (**MF_DISABLED**, **MF_ENABLED**, or **MF_GRAYED**) or -1 if not valid.

Parameters *nIDEnableItem* - Specifies the menu item to be enabled, as determined by *nEnable*. This parameter can specify pop-up menu items as well as standard menu items.

nEnable - Specifies the action to take. It can be a combination of **MF_DISABLED**, **MF_ENABLED**, or **MF_GRAYED**, with **MF_BYCOMMAND** or **MF_BYPOSITION**. These values can be combined by using the bitwise OR operator. These values have the following meanings: **MF_BYCOMMAND** Specifies that the parameter gives the command ID of the existing menu item. This is the default. **MF_BYPOSITION** Specifies that the parameter gives the position of the existing menu item. The first item is at position 0.

MF_DISABLED Disables the menu item so that it cannot be selected but does not dim it.

MF_ENABLED Enables the menu item so that it can be selected and restores it from its dimmed state. **MF_GRAYED** Disables the menu item so that it cannot be selected and dims it.

InsertMenuItem – insert a new element in the menu;

BOOL InsertMenu(UINT *nPosition*, UINT *nFlags*, UINT *nIDNewItem* = 0, LPCTSTR *lpszNewItem* = NULL);

BOOL InsertMenu(UINT *nPosition*, UINT *nFlags*, UINT *nIDNewItem*, const CBitmap* *pBmp*);

Return Value - Nonzero if the function is successful; otherwise 0.

Parameters: *nPosition* - Specifies the menu item before which the new menu item is to be inserted. The *nFlags* parameter can be used to interpret *nPosition* in the following ways:

MF_BYCOMMAND, MF_BYPOSITION

nFlags - Specifies how *nPosition* is interpreted and specifies information about the state of the new menu item when it is added to the menu. For a list of the flags that may be set. To specify more than one value, use the bitwise OR operator to combine them with the

MF_BYCOMMAND or **MF_BYPOSITION** flag.

nIDNewItem

Specifies either the command ID of the new menu item or, if *nFlags* is set to **MF_POPUP**, the menu handle (**HMENU**) of the pop-up menu. The *nIDNewItem* parameter is ignored (not needed) if *nFlags* is set to **MF_SEPARATOR**.

lpszNewItem

Specifies the content of the new menu item. *nFlags* can be used to interpret *lpszNewItem* in the following ways: **MF_OWNERDRAW, MF_STRING, MF_SEPARATOR**.

LoadMenu – **BOOL LoadMenu(LPCTSTR *lpszResourceName*);** **BOOL LoadMenu(UINT *nIDResource*);** - Return Value - Nonzero if the menu resource was loaded successfully; otherwise 0.

Parameters: *lpszResourceName* - Points to a null-terminated string that contains the name of the menu resource to load. *nIDResource* - Specifies the menu ID of the menu resource to load.

DeleteMenu – for deleting of an element from the menu.

AppendMenu- for appending of an element from the menu

CreateMenu – for creating of an element from the menu

ModifyMenu- for modifications of an element from the menu

DestroyMenu – for destroying of menu.

For example:

```
case IDM_CHECK: {HMENU hMenu = GetMenu(hWnd);
    UINT res = GetMenuState(hMenu, IDM_CHECK, MF_BYCOMMAND);
    if(res & MF_CHECKED)
        CheckMenuItem(hMenu, IDM_CHECK, MF_BYCOMMAND |
MF_UNCHECKED);
    else
        CheckMenuItem(hMenu, IDM_CHECK, MF_BYCOMMAND | MF_CHECKED); }
EnableMenuItem(hMenu, IDM_DISABLE, MF_BYCOMMAND | MF_ENABLED);
EnableMenuItem(hMenu, IDM_DISABLE, MF_BYCOMMAND | MF_GRAYED);}

case IDM_ADD:{HMENU hMenu = GetMenu(hWnd);
    if(GetMenuState(hMenu, IDM_ADD + 1, MF_BYCOMMAND) == -1) // or disable
    {
        MENUITEMINFO mii;
        ZeroMemory(&mii, sizeof(mii));
        mii.cbSize = sizeof(mii);
        mii.fMask = MIIM_ID | MIIM_TYPE | MIIM_STATE;
        mii.wID = IDM_ADD + 1;
        mii.fType = MFT_STRING;
        mii.dwTypeData = TEXT("Delete &me");
        mii.fState = MFS_ENABLED;
        InsertMenuItem(hMenu, IDM_ADD, FALSE, &mii); }
DeleteMenu(hMenu, IDM_ADD + 1, MF_BYCOMMAND);
case WM_RBUTTONDOWN:
    {
        HMENU hMenu = LoadMenu(hInst, MAKEINTRESOURCE(IDR_POP));
        HMENU hSubMenu = GetSubMenu(hMenu, 0);
        POINT pt = {LOWORD(IParam), HIWORD(IParam)};
        ClientToScreen(hWnd, &pt);
        TrackPopupMenu(hSubMenu, TPM_RIGHTBUTTON, pt.x, pt.y, 0, hWnd, NULL);
        DestroyMenu(hMenu); }
```

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder: : **Lesson3_0_win32_02(Menu)** (Click on **File | Open Workspace** | select file **menu.dsw**)
2. Look at the following component of the application carefully:
 - Choose Resource View and examine the two menus.
 - Open the file menu.cpp and concentrate your attention on the **switch (wmlId)** in case **WM_COMMAND** of **WndProc**.
 - Find applications of new functions.
 - Execute and try all functions of program, including the pressing of right button.
 - Trace the main logical connections.

TASK

1. Insert new ability “Ex_Menu” between “File” and “Help” in a new simple Application or in the application “menu”. The “Ex_Menu” includes the following abilities (A_O, (A_Ch, A_UnCh), (A_En, A_Gr), A_D, Exit)
2. Insert: Dialog box included edit box, connected with A_D function; a message which is connected with A_O; (A_Ch, A_UnCh) with check on not check methods; (A_En, A_Gr) with set enabled or grayed corresponding caption;
3. Insert changes in menu.cpp according to 1 and 2. Compile, Build and execute a new application.

LESSON 4

APPLICATION WITH DIALOG BOX WITH DIFFERENT DIALOG CONTROLS-1 (Calculation using integer and real data)

THEORY

Methods, used for work with Dialog Box with different Dialog Controls

New Methods and functions in the lesson example are:

➤ `BOOL *LPt=NULL; BOOL SIG=TRUE;`
`int GetDlgItemInt(hDlg, edit box identification ,LPt, SIG) ;` this function retrieves integer number which insert as text in edit box;

➤ `SetDlgItemInt(hDlg, edit box identification, INumber,TRUE);` shows the INumber - integer number in edit box.

for example: `BOOL *LPt=NULL; BOOL SIG=TRUE;`

`g1=GetDlgItemInt(hDlg, IDC_EDIT1, LPt, SIG); SetDlgItemInt(hDlg, IDC_EDIT4, g1, TRUE);`

Converting functions:

➤ `double variable=strtod(char var[10],&stopstring);` - this function converts some text which insert in edit box to double number; (char *stopstring;)

➤ Function **strol** converts some text to long.

➤ `int variable=atoi(Text)` – converts text to integer number.

➤ Function **atol** converts text to long number.

➤ Function **atof** converts text to double number.

➤ `gcvt(DNumber,10,Text);` this function retrieves as parameter the variable -Text like the double DNumber ;

➤ `fcvt(FNumber,10,Text);` this function retrieves as parameter the variable -Text like the float number FNumber ;

For example: `char t1[10]; GetDlgItemText(hDlg, IDC_EDIT1, t1, 10);`

`x1=strtod(t1,&stopstring); gcvt(x1,10,t11);`

`MessageBox(hDlg, t11, "OK", MB_OK); SetDlgItemText(hDlg, IDC_EDIT3, t11);`

INSTRUCTION

1. Start application from folder: **Lesson4_0**. Execute and note all it abilities.

2. Look at following component of application carefully:

➤ Resources and their properties (Menu and Dialogs)

➤ In main look for new functions.

➤ You have to pay attention to:

// Global Variables ; // Forward declarations; LRESULT CALLBACK WndProc.....

RESULT CALLBACK MOITAR(..... RESULT CALLBACK MOITAI(.....

TASK

1. Make in your copy of example next changes:

➤ Add a new ability in menu "Calculation"

➤ Add a new dialog connected with the new menu function. In the Dialog insert 3 edit boxes and 3 corresponding buttons.

➤ Add new function, which multiplies number from the first edit box by second number. The Result has to show in the third edit box.

➤ Prepare and Execute new application.

LESSON 5

APPLICATION WITH DIALOG BOX WITH DIFFERENT DIALOG CONTROLS 2 (Combo, List and Group Controls)

THEORY

Methods, used for work with Dialog Box with different Dialog Controls

New Methods and functions in the lesson example are:

SendDlgItemMessage(hDlg, ID_of_DialogControl, Processing_Way, Index, (LPARAM) "Some text in Dialog Control") This function send in different way some text in someone Dialog Control. Specifies the value returned by the control's window procedure, or 0 if the control was not found.

Parameters are: **ID_of_DialogControl** – Identification of Dialog Control;

Processing_Way – It depends on the mode of Dialog Controls and processing type:

- for **list box** the standard constant names begin with **LB**: **LB_GETTEXT**, **LB_SETTEXT**, **LB_RESETCONTENT**, **LB_INSERTSTRING**, **LB_SELECTSTRING**, **LB_FINDSTRING** **LB_SETCURSEL**, **LB_GETCURSEL**;
- for **combo box** the standard constant name begin with **CB**: **CB_GETTEXT**, **CB_SETTEXT**, **LB_INSERTSTRING**, **CB_ADDSTRING**, **CB_SELECTSTRING**, **CB_GETCURSEL**, **CB_SETCURSEL**, **CB_GETLBTEXT**, **CB_FINDSTRING** ;
- for **radio** and **check** boxes the standard constant names begin with **BM**: **BM_SETCHECK**, **BM_GETCHECK**;
- for **edit box** the standard constant names begin with **WM**: **WM_GETTEXT**, **WM_SETTEXT**.

for example - `SendMessage(hEdit, WM_SETTEXT, 0, (LPARAM) "Some data");`

- for **progress bar** the standard constant names begin with **PBM**: **PBM_SETRANGE**, **PBM_DELTAPOS**, **PBM_SETSTEP**, **PBM_SETPOS**, **PBM_DELTAPOS**;

Index :

a/ Set position of sending text in the Dialog Control in cases of list and combo boxes.

b/ Set Check or not check in cases of radio and check boxes using the standard constant : **BST_CHECKED**, **BST_UNCHECKED**.

c/ Set the text size in case of Edit box.

(LPARAM) "Some text in Dialog Control" – Specifies additional message-dependent information for example sending text or text variable.

For example:

```
cbIndex=SendDlgItemMessage(hDlg,IDC_COMBO1,CB_GETCURSEL,0,0);
SendDlgItemMessage(hDlg,IDC_COMBO1,CB_FINDSTRING,0,(LPARAM)data)
c_index=SendDlgItemMessage(hDlg,IDC_COMBO1,CB_ADDSTRING,0,(LPARAM) "Row 3
in Combo Box");
SendDlgItemMessage(hDlg,IDC_LIST1,LB_SETCURSEL,0,0);
lbIndex=SendDlgItemMessage(hDlg,IDC_LIST1,LB_GETCURSEL,0,0);
SendDlgItemMessage(hDlg,IDC_LIST1,LB_FINDSTRING,0,(LPARAM)data)
SendMessage(hEdit, WM_SETTEXT, 0, (LPARAM) "Some Text");
```

IsDlgButtonChecked(hDlg, ID_CHECK) -retrives the value according either the box (ID_CHECK) is checked or not.

ChechDlgButton(hDlg, ID_CHECK, (BST_CHECKED or BST_UNCHECKED)) change the status of box.

`int check_or_not =IsDlgButtonChecked(hDlg,ID_CHECK)` -ret rives the value according either the box (ID_CHECK) is checked or not.

EnableWindow(GetDlgItem(hDlg, IDC_B), state); Indicates the state before the **EnableWindow** member function was called. The return value is nonzero if the window was previously disabled. The return value is 0 if the window was previously enabled or an error occurred.

An Example for a group from 3 radio buttons.

```
        CheckDlgButton(hDlg, IDC_A, BST_CHECKED); // => CheckRadioButton(...)
{UINT      state      =      IsDlgButtonChecked(hDlg,      IDC_CHECK_CTRL);      //
SendMessage(<BM_GETCHECK>);
        EnableWindow(GetDlgItem(hDlg, IDC_A), state);
        EnableWindow(GetDlgItem(hDlg, IDC_B), state);
        EnableWindow(GetDlgItem(hDlg, IDC_C), state);}
```

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder **Lesson5_0**. Execute end note all it abilities. Pay to attention to abilities “Dialog Sequence 1” and “Dialog Sequence 2”. Look at the way of processing of messages from message boxes.
2. Pay to attention in ability “New Dialog”. You can try to insert the same text twice in combo box and / or list box. Note what is happened if the edit box is empty.
3. Look at following components of application carefully:
 - Controls of the Dialog IDD_NewDiag and their properties
 - Look at the **func_NewDiag**.
 - You have to pay attention on files f_diag.cpp and a.cpp for:
// Global Variables ; // Forward declarations; LRESULT CALLBACK WndProc.....
Note in file func_NewDiag -> RESULT CALLBACK func_NewDiag (....)
3. Examine function **func_NewDiag**.
4. Find all operators, which correspond with initialization of controls; checking of contains of boxes for input text. Getting and setting of cursor and sending of input data in the check box.
4. Open, execute end examine application from folder : **Lab5Examplewin32_04(Dialog)**. You can focus your attention in function Dlg.cpp in part “init group1” This is an example for grouping of radio buttons.

TASK

1. Make a copy of the example folder
2. Make following changes: Insert in Main menu “ My final dialog” and create new Dialog Box (D Last) connected with it. In “D Last”, include a combo box and list box.
3. Add a new function in a new file, which work with new Dialog Box. The file name has to be my_dialog.cpp.
Set four rows text to combo and list boxes during the initializations. At first let there are lists of texts in combo and in list box. Provide function with abilities to select data from boxes and send messages with selected data.
4. Prepare and Execute new application.
5. Insert some changes (new button, new kind of message) in part of program, which is connected with ability “Sequence Dialog”. Verify the last version of your program.

LESSON 6

Application with Dialog Box (MOUSE MOVING TIMER, PROGRESS BAR)

THEORY

New methods and functions

- **InitCommonControls()** – initialization of controls ;
- **int LOWORD(IParam)** – returns the x-coordinate of mouse; **int y = HIWORD(IParam)** returns the y-coordinate of mouse;
- **SetTimer(hDlg, TIMER1, 20, NULL)** – creates timer and set the start value and changing step of it variable .
- **KillTimer(hDlg, TIMER1)** – destroys the timer.
- New application for progress bar of function:
SendDlgItemMessage(hDlg, IDC_PROGBAR, PBM_SETRANGE, 0, MAKELPARAM(0, 1000));

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder **Lesson6_1mywin32**. Execute end note all it abilities. Pay to attention in ability "Show Dialog".
2. Look at following component of application carefully: Dialog **IDD_DIALOG3** and its controls.
 - In **mywin32.cpp** look for the function **functionMouse**.
 - You have to pay attention in file to: **#include "supplementary.h"**
 - **#define MAX_LOADSTRING 100**
 - Note in file **207-02-20.cpp** - -> **#define TIMER1 500**

The main points are:

- **for ménage of mouse:**

```
case WM_MOUSEMOVE: {int x = LOWORD(IParam); int y = HIWORD(IParam);  
SetDlgItemInt(hDlg, IDC_MOUSEX, x, TRUE); SetDlgItemInt(hDlg, IDC_MOUSEY, y,  
TRUE); break;}
```

- **for manage of timer:**

```
if (LOWORD(wParam) == IDC_TSTART) {SetTimer(hDlg, TIMER1, 20, NULL); break;}  
if (LOWORD(wParam) == IDC_TSTOP) {KillTimer(hDlg, TIMER1); SetDlgItemInt(hDlg,  
IDC_TIMER, 0, NULL); break;}
```

- **for showin of progress bar:**

```
SendDlgItemMessage(hDlg, IDC_PROGBAR, PBM_SETRANGE, 0, MAKELPARAM(0,  
1000));
```

```
SendDlgItemMessage(hDlg, IDC_PROGBAR, PBM_DELTAPOS, 100, 0); .....
```

```
SendDlgItemMessage(hDlg, IDC_PROGBAR, PBM_SETSTEP, step, 0);
```

```
SendDlgItemMessage(hDlg, IDC_PROGBAR, PBM_SETPOS, 900, 0);
```

```
SendDlgItemMessage(hDlg, IDC_PROGBAR, PBM_DELTAPOS, -500, 0);
```

TASK

1. Make a copy of the example folder.
2. Make following changes:
 - Add new abilities of menu "My Mouse" and "My Timer Task".
 - Create new Dialogs which are connected with these abilities. Let these dialogs include some edit boxes, their corresponding button and progress bar.
 - Write functions for processing of this dialogs. At first include only initialization. At second include function which get and send data. At last, you can include their show with progress bar.
3. Built and execute your new application variant.

LESSON 7

Graphical Application –Introduction

THEORY

<u>CreateSolidBrush</u>	Initializes a brush with the specified solid color.
<u>CreateHatchBrush</u>	Initializes a brush with the specified hatched pattern and color.
<u>CreateBrushIndirect</u>	Initializes a brush with the style, color, and pattern specified in a <u>LOGBRUSH</u> structure. (lbColor , lbHatch , lbStyle)
<u>CreatePatternBrush</u>	Initializes a brush with a pattern specified by a bitmap .
<u>CreateDIBPatternBrush</u>	Initializes a brush with a pattern specified by a device-independent bitmap (DIB) .
<u>CreateSysColorBrush</u>	Creates a brush that is the default system color.

New methods and functions

➤ Usually used

hdc = **BeginPaint**(hWnd, &ps); where PAINTSTRUCT ps; **EndPaint**(hWnd, &ps);
hdc=**GetDC**(hWnd); // handle device context
RECT rt; // type definition (RECT is standard type).
GetClientRect(hWnd, &rt); // getting of parameters of current rectangle in rt. rt.left, rt.right, rt.top, rt.bottom (integer values)

➤ **int SetROP2(int nDrawMode);** sets drawing mode: R2_BLACK,...

➤ **Colors** (COLORREF **crColor** is a parameter which is used in graphical functions)
RGB(red components, green components, blue component) The components are in interval (from 0 to 255)

Examples: red - RGB(255,0,0), white -RGB(255,255,255), yellow - RGB(255,102, 228), brown - RGB(255,200,100)

➤ HBITMAP hBmp=**LoadBitmap**(hInst, (const char*)IDB_BITMAP2); load like color pattern.

➤ Outputting text

DrawText(hdc, szHello, strlen(szHello), &rt, DT_CENTER) // draw text (szHello) from to first row of rt in the way DT_CENTER;

SetTextColor(hdc, RGB(190, 190, 190)); // set color text in handle **hdc**;

SetBkColor(hdc, RGB(100, 100, 100)); // set color around the text

TextOut(hdc, 600, 300, "text1", strlen("text1")); // show text ("text1"), from position with coordinates (600;300) in handle **hdc**.

➤ Painting tool

SelectObject(hdc, hPen); or **SelectObject**(hdc, hB); // select an object hPen or hB in handle hdc.

DeleteObject(hPen); // disengage object.

BOOL CreatePen(int nPenStyle, int nWidth, COLORREF crColor); The first version of **CreatePen** initializes a pen with the specified style, width, and color. The pen can be subsequently selected as the current pen for any device context.

HPEN hPen = **CreatePen**(PS_SOLID, 2, **RGB(0, 0, 0)**); // Create and **Set Pen**

The pen can subsequently be selected as the current brush for any device context.

SelectObject(hdc, hPen); // select object hPen in handle hd

BOOL CreateSolidBrush(COLORREF crColor); Initializes a brush with a specified solid color. The brush can subsequently be selected as the current brush for any device context.

```
HBRUSH hBrush = CreateSolidBrush( RGB(132, 123, 231) ); // filling color
SelectObject(hdc, hBrush);
```

BOOL CreateHatchBrush(int nIndex, COLORREF crColor); Initializes a brush with the specified hatched pattern and color. The brush can subsequently be selected as the current brush for any device context.

Value of **nIndex** may be following constants: **HS_BDIAGONAL** Downward hatch (left to right) at 45 degrees, **HS_CROSS** Horizontal and vertical crosshatch, **HS_DIAGCROSS** Crosshatch at 45 degrees, **HS_FDIAGONAL** Upward hatch (left to right) at 45 degrees, **HS_HORIZONTAL** Horizontal hatch, **HS_VERTICAL** Vertical hatch

BOOL CreatePatternBrush(CBitmap* pBitmap); Initializes a brush with a pattern specified by a bitmap. The brush can subsequently be selected for any device context that supports raster operations. The bitmap identified by *pBitmap* is typically initialized.

The programmer can use his own fill set with a resource **bitmap**. In following example this fill set by resource with ID - IDB_BITMAP1.

```
HBITMAP hBmp=LoadBitmap(hInst, (const char*)IDB_BITMAP1); // loading of
bitmap image in hBmp;
```

```
HBRUSH hBrush2=CreatePatternBrush(hBmp); /// user filling
```

➤ **Painting of primitives**

Rectangle(hdc, Xleft, Ytop, Xright, Ybottom) // drawing a **bar** set with its 2 corner points
for example - Rectangle(hdc, 120, 200, 250, 400);

Ellipse(hdc, Xcenter, Ycenter, RadiusX, RadiusY); // drawing a **ellipse** set with its centre and radiuses.
for example - Ellipse(hdc, 300, 150, 400, 500);

MoveToEx (hdc, X, Y, NULL); // Moving the graphical cursor without trace to a point(X,Y);

LineTo(hdc, X,Y); // Drawing a line from position of graphical cursor to a point (X,Y).
for example - LineTo(hdc, rt.right-10, rt.bottom-10);

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder **Lesson7_0(PaintDraw)**. Execute and note all its abilities. Pay attention in main window: its text, figures, colors. Try to paint something (line, curve, rectangle) with Left and Right Button of the mouse.

2. Look at following component of application carefully. Resources and their properties (Bitmap).

3. You have to pay attention in file **second.cpp** to:

➤ **for Statical Painting**

➤ **Interactive Painting** with Left or Right Button of Mouse

4. Look at the rest examples in folder **Lab7_Draw1**.

TASK

1. Make a copy of the folder for the example.

2. Make following changes:

- Create a new bitmap and use it instead of the existing resource.
- Output some text after changing of color of symbols and its background.
- Create new pen. Paint a triangle by painting 3 lines.

4. Build and execute your new application variant.

5.

LESSON 8

Graphical Application – Moving with mouse (Drag&Drop)

THEORY

New methods and functions

Declaration – new points

static Rect rect1; static Rect* sel; static POINTS anchor;

enum [tag] {enum-list} [declarator]; // for definition of enumerated type

Methods and functions

MAKEPOINTS(IParam) –return point values, for example anchor= MAKEPOINTS(IParam)
(anhor.x = LOWORD(Iparam))

SetCapture(hWnd) - If the control is activated and windowless, this function causes the control's container window to take possession of the mouse capture, on the control's behalf. Otherwise, this function causes the control itself to take possession of the mouse capture.

ReleaseDC(hWnd, hdc); Call this function to release the display device context of a container of a windowless control, freeing the device context for use by other applications. The application must call **ReleaseDC** for each call to GetDC.

Calling class methods Rect

```
rect1.Draw(hdc)
sel = rect2.HitTest(MAKEPOINTS(IParam))
sel->Draw(hdc);
sel->DrawAsSelected(hdc);
sel->MoveFromTo(anchor, MAKEPOINTS(IParam));
```

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder **Lab8_Move**. Execute end select some rectangle and move it.
2. Look at following component of application carefully:
 - File win32_7.h – Class Rect. Note its methods.
 - Find in WndProc drawing of rectangles.
 - Trace the way of processing of selecting of some rectangle.
 - Explain what is happened when: press left button down, mouse move, left button up

TASK

1. Make a copy of the folder for the example.
2. Make following changes:
 - insert new pen and brush;
 - show fill rectangles with new coordinates;
 - Try to make moving of new rectangle.
3. Built and execute your new application variant.

LESSON 9

Text Editor (WIN32 and using MFC)

THEORY

New methods and functions

Let's have the declarations: static HWND **hEdit**; static POINT **ptOld**;

➤ The Method **CreateWindow** "**EDIT**" with following properties using handle hEdit.

hEdit = CreateWindow("EDIT", NULL, WS_CHILD | WS_VISIBLE | WS_VSCROLL | ES_LEFT | ES_MULTILINE | ES_AUTOVSCROLL, 0, 0, 0, 0, hWnd, NULL, hInst, NULL);

➤ The Method **GetCursorPos** returns the cursor position in variable **ptOld**.

for example: GetCursorPos(&ptOld);

➤ The method **SendMessage** returns the result of the message processing; its value depends on the message sent.

LRESULT SendMessage(hEdit, UINT message, WPARAM wParam = 0, LPARAM lParam = 0) Parameters:

message - Specifies the message to be sent.

WM_Message: WM_UNDO, WM_CUT, WM_COPY, WM_PASTE, WM_CLEAR, WM_SETTEXT

EM_Message: EM_CANUNDO, EM_SETSEL;

wParam - Specifies additional message-dependent information.

lParam - Specifies additional message-dependent information.

for instance: SendMessage(hEdit, WM_CUT, 0, 0);

SendMessage(hEdit, WM_COPY, 0, 0);

SendMessage(hEdit, WM_PASTE, 0, 0);

SendMessage(hEdit, WM_CLEAR, 0, 0);

SendMessage(hEdit, WM_SETFONT, (WPARAM) hFont, TRUE);

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder **Lab9_Editor\Lesson9_0**. Execute application and note all its abilities. Pay your attention to main window and its ability to edit text, including "format". Try to use all its possibilities.

2. Look at following component of application carefully:

➤ Processing of edition messages. Find examples for the new functions in the example.

➤ Look at operators which realize ability for the font changing.

3. Create new Application- Text Editor with MFC

➤ File-New Project->MFC : insert some name of project: for example "**abcd**" -> confirm.

➤ From abilities of this step 1 choose the kind of application "Multiple Documents" -> "Next";

➤ From abilities of this step 2 choose file multitude of application "None using file" -> "Next";

➤ From abilities of this step 3 choose Active X controls of application "None using " -> "Next";

➤ Next two steps 4 and 5 examine standard choices and continue.

➤ In the last step 6 you have to look at in box "Class name". The class of application is "**CAbcdView**". In combo box "Base Class" you have to choose "**cEditView**" and can finish.

➤ Build and execute your application.

TASK

1. Make a copy of the folder for the example.

2. Make following changes:

➤ Main changes in menu and corresponding changes in WndProc;

➤ Try to exclude something from properties of CreateWindow;

➤ Insert Message a few boxes in these cases.

➤ Build and execute your new application variant.

3. Create new text editor application using MFC.

LESSON 10 (Open and Save)

THEORY

New methods and functions

1. These methods require the following files:

```
#include <stdio.h>
#include <CommDlg.h> // for using of standart dialog
#include <fstream> // for working with files
#include <string>
```

2. Opening and saving of files.

void open(const char* szName, int nMode = ios::out, int nProt = filebuf::openprot);

Parameters: *szName* The name of the file to be opened during construction.

nMode- An integer containing mode bits defined as **ios** enumerators that can be combined with the OR (|) operator. See the **ofstream** constructor for a list of the enumerators. The **ios::out** mode is implied.

nProt- The file protection specification; defaults to the static integer **filebuf::openprot**. See the **ofstream** constructor for a list of the other allowed values.

void close();

3. Next function returns the value of *dest*. Sets buffers to a specified character.

void *memset(void *dest, int c, size_t count);

for example: **OPENFILENAME** ofn;

memset(&ofn, 0, sizeof(OPENFILENAME)); (it requires directive "#include <string>")

Note the filling of fields of ofn ----

ofn.nMaxFileTitle = sizeof(szFileTitle); // chosen file name

ofn.lpstrFileTitle = (LPSTR)szFileTitle; // path of file

ofn.lpTemplateName = MAKEINTRESOURCE(IDD_OPEN); // IDD_Open is user Dialog

4. Functions for getting of file name we want to open or save: **GetOpenFileName**, **GetSaveFileName**

for example :**GetOpenFileName(&ofn)==TRUE** **GetSaveFileName(&ofn)==TRUE**

5. The main example of lesson includes well known function "sprintf". In this case the function create in the variable text string - **sprintf** (variable, "Some text %s Another text %s", a,b);

for example: **char szTemp[256];**

sprintf(szTemp, "Selected file is %s (full path: %s)", ofn.lpstrFileTitle,

ofn.lpstrFile);

MessageBox(hWnd, szTemp, "Info", MB_OK|MB_ICONINFORMATION);

INSTRUCTION

1. Start MS Visual C++ and Open Workspace of project from folder Lab10_FileOpenSave\Lesson10_0. Execute application end note all it abilities. Pay to attention to main window and it ability to edit text. Try to open and to save a file.

2. Look at following component of application carefully: the dialog IDD_OPEN and it's controls;

-functions **OnOpenFile(HWND hWnd)** and **OnSaveFile(HWND hWnd)**.

3. Note the part of program text that provide the ability to edit some text.

TASK

1. Make a copy of the folder for the example.

2. Make: changes the default file name and file type; include additional information in message boxes such as: **ofn.lpstrFilter**, **ofn.nMaxFile**, **ofn.lStructSize**

3. Add some new controls in the dialog IDD_OPEN. Build and execute your new application variant.